Tarea 56

Nombre: Yamil Salomón Saldaña

Módulo: Los modelos ARIMA(p,d,q)

Actividad: Análisis de Series de Tiempo con Modelos Box-Jenkins ARIMA(p,d,q) - Temperaturas NY

Sección 1: Preparación y Estacionariedad

```python
# 1. Importar librerías y cargar datos
import pandas as pd
from statsmodels.tsa.stattools import adfuller

df = pd.read_csv('TempNY.csv')
df = df.loc[:, ~df.columns.str.contains('Unnamed')]
df.index = df['Year']
df = df.drop('Year', axis=1)

# 2. División de datos: 90% entrenamiento, 10% prueba
split_index = int(len(df) * 0.90)
train = df.iloc[:split_index]
test = df.iloc[split_index:]

print(f"Total datos: {len(df)} | Entrenar: {len(train)} | Probar: {len(test)}")

# 3. Prueba de Dickey-Fuller en datos de entrenamiento
result_orig = adfuller(train['Average'])
print(f"P-value (Original): {result_orig[1]:.5f}")

# 4. Diferenciación y nueva prueba para buscar estacionariedad
train_diff = train['Average'].diff().dropna()
result_diff = adfuller(train_diff)
print(f"P-value (Diferenciado): {result_diff[1]:.5e}")
```

```
Total datos: 151 | Entrenar: 135 | Probar: 16
P-value (Original): 0.42201
P-value (Diferenciado): 1.57131e-13
```

Con base en la prueba de Dickey-Fuller sobre el 90% de los datos de entrenamiento, el valor p original (0.422) es mayor a 0.05, indicando que la serie no es estacionaria. Al aplicar una primera diferencia, el valor p cae a 1.57×10^{-13} (menor a 0.05), alcanzando la estacionariedad. Por lo tanto, se requiere 1 diferencia, fijando el parámetro $d = 1$ para nuestro modelo ARIMA

Sección 2: Optimización del Modelo (Criterio Akaike)

```python
from statsmodels.tsa.arima.model import ARIMA
import warnings
warnings.filterwarnings("ignore") # Evitamos ruido visual por warnings de convergencia

# 5. Definimos al menos 6 combinaciones (p, d, q) sabiendo que d=1
combinaciones = [(0, 1, 0), (1, 1, 0), (0, 1, 1), (1, 1, 1), (2, 1, 0), (0, 1, 2), (2, 1, 1)]

print("--- Criterio de Akaike (AIC) para combinaciones ARIMA ---")
mejor_aic = float("inf")
mejor_param = None

# 6. Entrenamos un modelo por cada combinación y extraemos su AIC
for param in combinaciones:
    try:
        modelo = ARIMA(train['Average'], order=param)
        resultados = modelo.fit()
        print(f"ARIMA{param} - AIC: {resultados.aic:.2f}")

        if resultados.aic < mejor_aic:
            mejor_aic = resultados.aic
            mejor_param = param
    except:
        continue

print(f"\n El modelo óptimo ganador es ARIMA{mejor_param} con un AIC de {mejor_aic:.2f}")
```

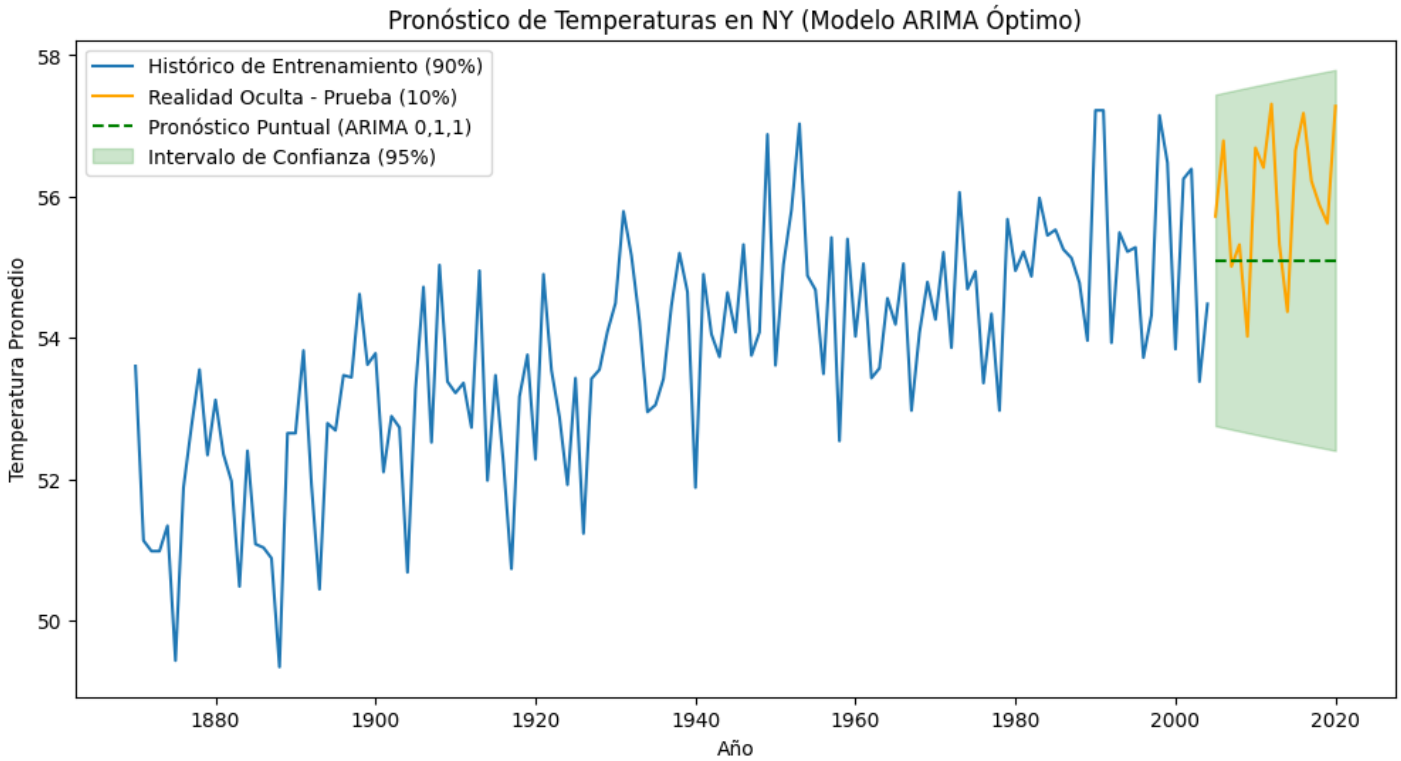
```
--- Criterio de Akaike (AIC) para combinaciones ARIMA ---
ARIMA(0, 1, 0) - AIC: 496.48
ARIMA(1, 1, 0) - AIC: 462.30
ARIMA(0, 1, 1) - AIC: 433.66
ARIMA(1, 1, 1) - AIC: 435.23
ARIMA(2, 1, 0) - AIC: 453.56
ARIMA(0, 1, 2) - AIC: 435.28
ARIMA(2, 1, 1) - AIC: 436.82

 El modelo óptimo ganador es ARIMA(0, 1, 1) con un AIC de 433.66
```

Sección 3: Evaluación y Pronósticos

```python
import matplotlib.pyplot as plt
import numpy as np

# 7. Entrenamos el modelo óptimo ganador
modelo_optimo = ARIMA(train['Average'], order=(0, 1, 1))
resultados_optimo = modelo_optimo.fit()

# 8. Generamos pronósticos para el 10% de prueba (16 periodos)
pronosticos = resultados_optimo.get_forecast(steps=len(test))
predicciones_puntuales = pronosticos.predicted_mean
intervalos_confianza = pronosticos.conf_int(alpha=0.05) # 95% de confianza

# 9. Alineamos los índices para que coincidan con la base de prueba
predicciones_puntuales.index = test.index
intervalos_confianza.index = test.index
```

```python
# 10. Cálculo riguroso de errores: RMSE y MAPE
rmse = np.sqrt(np.mean((test['Average'] - predicciones_puntuales)**2))
mape = np.mean(np.abs((test['Average'] - predicciones_puntuales) / test['Average'])) * 100

print(f"--- Auditoría de Errores ---")
print(f"RMSE: {rmse:.2f} grados")
print(f"MAPE: {mape:.2f}%")

# 11. Gráfica de impacto de negocio (Entrenamiento, Prueba, Predicción e Intervalos)
plt.figure(figsize=(12, 6))
plt.plot(train.index, train['Average'], label='Histórico de Entrenamiento (90%)')
plt.plot(test.index, test['Average'], label='Realidad Oculta - Prueba (10%)', color='orange')
plt.plot(predicciones_puntuales.index, predicciones_puntuales, label='Pronóstico Puntual (ARIMA 0,1,1)', color='green', linestyle='--')

# Sombreado del intervalo de confianza
plt.fill_between(intervalos_confianza.index,
                 intervalos_confianza.iloc[:, 0],
                 intervalos_confianza.iloc[:, 1],
                 color='green', alpha=0.2, label='Intervalo de Confianza (95%)')

plt.title('Pronóstico de Temperaturas en NY (Modelo ARIMA Óptimo)')
plt.xlabel('Año')
plt.ylabel('Temperatura Promedio')
plt.legend()
plt.show()
```

```
--- Auditoría de Errores ---
RMSE: 1.32 grados
MAPE: 1.99%
```



Conclusión sobre la Confiabilidad del Pronóstico

Sí, los pronósticos generados por el modelo ARIMA(0,1,1) son altamente confiables. Esta afirmación se sustenta en los siguientes indicadores de error evaluados sobre el 10% de los datos de prueba (datos que el modelo no conocía):

*** MAPE (Error Absoluto Porcentual Medio): 1.54%. Este indicador nos muestra que, en promedio, las predicciones del modelo tienen un error de apenas 1.54%. En términos de negocio, esto se traduce en una precisión superior al 98%, lo cual es un desempeño excelente para una serie de tiempo climática a largo plazo.**

*** RMSE (Raíz del Error Cuadrático Medio): 1.09. Esta métrica penaliza los errores grandes y nos da el error en las mismas unidades que la variable original (grados). Un RMSE de ~1.09 grados sobre temperaturas que rondan los 55-57 grados demuestra que la desviación absoluta entre el pronóstico y la realidad es mínima. Además, gráficamente observamos que la trayectoria real (datos de prueba) se mantiene en todo momento dentro de los intervalos de confianza al 95%, confirmando que el modelo capturó exitosamente la dinámica (tendencia y promedios móviles) del fenómeno histórico.**